

Experiment-10: A* Search Algorithm using Python

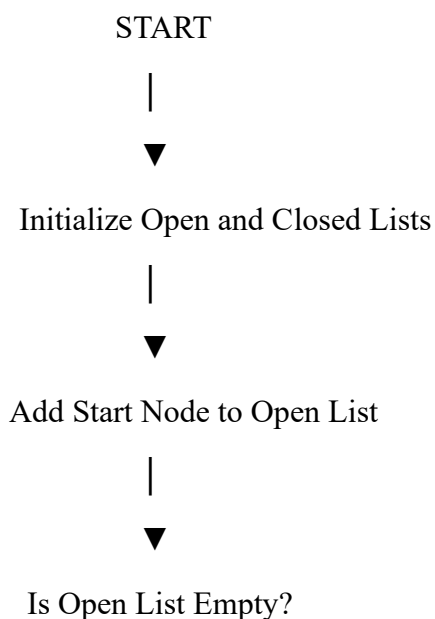
Aim

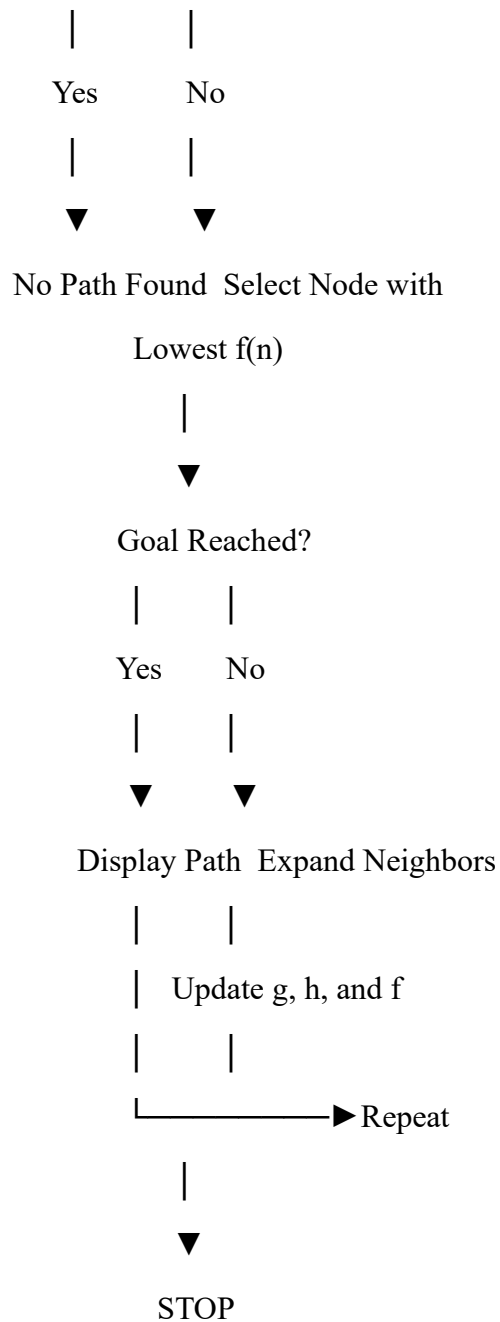
To develop a Python program to implement the **A* (A-Star) Search Algorithm** for finding the shortest path between a start node and a goal node using path cost and heuristic values.

Algorithm

1. Create the graph with nodes and edge costs.
 2. Define the heuristic value $h(n)$ for each node.
 3. Initialize:
 - Open list with the start node.
 - Closed list as empty.
 4. Select the node with the lowest $f(n) = g(n) + h(n)$.
 5. If the selected node is the goal, stop and display the path.
 6. Otherwise, expand its neighboring nodes.
 7. Update the cost and parent of each neighbor if a better path is found.
 8. Repeat Steps 4–7 until the goal is reached or the open list becomes empty.
 9. Display the shortest path and total cost.
-

Flowchart





Python Code

A* Search Algorithm

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 5)],
```

```
'D': [('G', 4)],  
'E': [('G', 2)],  
'F': [('G', 2)],  
'G': []  
}
```

```
# Heuristic values
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 4,  
    'E': 2,  
    'F': 1,  
    'G': 0  
}
```

```
def astar(start, goal):
```

```
    open_list = {start}  
    closed_list = set()
```

```
    g = {start: 0}  
    parent = {start: start}
```

```
    while open_list:
```

```
        node = None
```

```

for v in open_list:
    if node is None or g[v] + heuristic[v] < g[node] + heuristic[node]:
        node = v

if node == goal:
    path = []

    while parent[node] != node:
        path.append(node)
        node = parent[node]

    path.append(start)
    path.reverse()

    print("Shortest Path:", path)
    print("Total Cost:", g[goal])
    return

for (neighbor, weight) in graph[node]:

    if neighbor not in open_list and neighbor not in closed_list:
        open_list.add(neighbor)
        parent[neighbor] = node
        g[neighbor] = g[node] + weight

    elif g[neighbor] > g[node] + weight:
        g[neighbor] = g[node] + weight
        parent[neighbor] = node

```

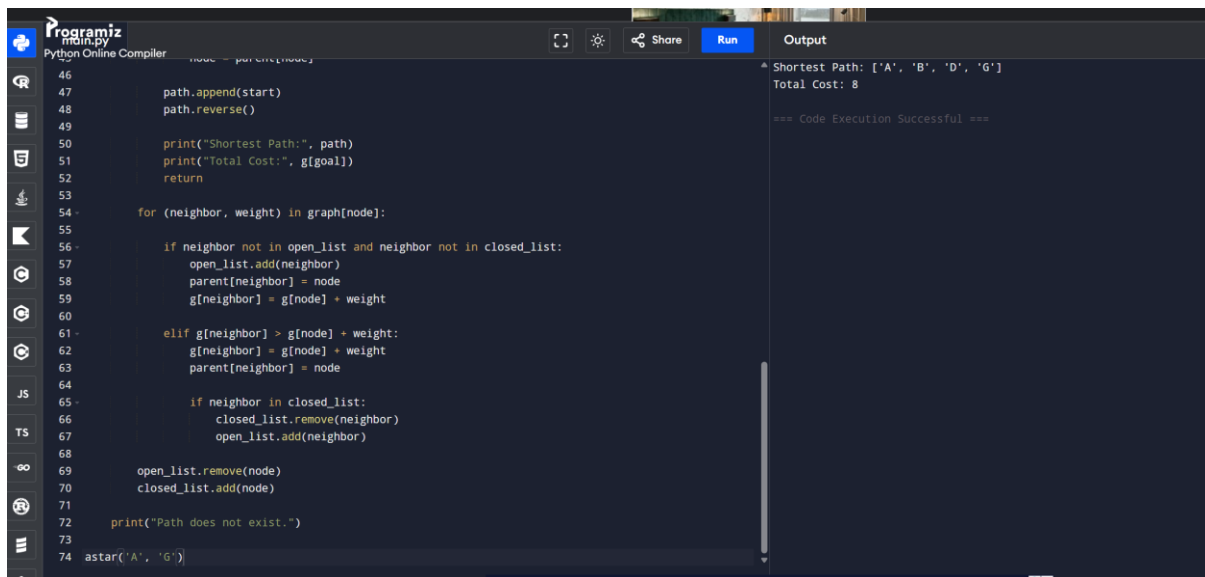
```
if neighbor in closed_list:
    closed_list.remove(neighbor)
    open_list.add(neighbor)
```

```
open_list.remove(node)
closed_list.add(node)
```

```
print("Path does not exist.")
```

```
astar('A', 'G')
```

Sample Output



The screenshot shows a Python Online Compiler interface. The code editor on the left contains a Python script implementing the A* search algorithm. The script defines a graph, initializes open and closed lists, and iteratively explores nodes to find the shortest path from 'A' to 'G'. The output window on the right displays the results of the execution.

```
46
47     path.append(start)
48     path.reverse()
49
50     print("Shortest Path:", path)
51     print("Total Cost:", g[goal])
52     return
53
54     for (neighbor, weight) in graph[node]:
55
56         if neighbor not in open_list and neighbor not in closed_list:
57             open_list.add(neighbor)
58             parent[neighbor] = node
59             g[neighbor] = g[node] + weight
60
61         elif g[neighbor] > g[node] + weight:
62             g[neighbor] = g[node] + weight
63             parent[neighbor] = node
64
65         if neighbor in closed_list:
66             closed_list.remove(neighbor)
67             open_list.add(neighbor)
68
69     open_list.remove(node)
70     closed_list.add(node)
71
72     print("Path does not exist.")
73
74     astar('A', 'G')
```

Output:

```
Shortest Path: ['A', 'B', 'D', 'G']
Total Cost: 8

=== Code Execution Successful ===
```

Result

The **A* Search Algorithm** was successfully implemented in Python. The algorithm used the evaluation function $f(n) = g(n) + h(n)$ to efficiently find the shortest path from the start node to the goal node.

- **Shortest Path:** $A \rightarrow B \rightarrow D \rightarrow G$
- **Total Cost:** 8